



Internship Task No: 01

Domain: Machine Learning

Topic: Heart Disease Prediction System

Submitted to: Digital Empowerment Network

Submitted by: Muhammad Sarmad Usman

Student ID: CID STU-ML-251-555

Submission Date: July 23, 2025

Contents

 Problem Statement	3
 Objective	3
 Dataset	3
 Tools & Technologies Used	3
✓ Step 0: Environment Setup (Library Imports):	4
✓ Step 1: Dataset Selection:	5
✓ Step 2: Data Understanding & Preprocessing (Part 1):	6
✓ Step 2: Data Understanding & Preprocessing (Part 2):	7
✓ Step 2: Data Understanding & Preprocessing (Part 3):	8
✓ Step 3: Exploratory Data Analysis (EDA):	9
✓ Step 4: Feature Scaling & Train-Test Split:	11
✓ Step 5: Model Building & Evaluation:	12
◆ 1. Logistic Regression Training and Evaluation:	12
◆ 2. Random Forest Training and Evaluation:	13
✓ Step 6: Model Evaluation Visualization:	14
◆ 1. Confusion Matrix:	14
 2. Accuracy Comparison Chart:	15
✓ Step 7: Deployment with Streamlit + PDF Report Generation:	16
✓ Step 7.1:  Streamlit App – User Interface:	21
✓ Step 7.2:  Generated PDF Report – Output Preview:	23
✓ Step 8: Final Summary and Conclusion:	24

Heart Disease Prediction Using Machine Learning

Problem Statement

Heart disease is a leading cause of death globally. Early prediction and diagnosis can save lives, reduce medical costs, and assist doctors in making informed decisions. The goal of this project is to build a reliable machine learning system that predicts whether a person is likely to have heart disease based on various medical attributes.

Objective

To implement a complete machine learning workflow on a real-world dataset for heart disease prediction. This includes:

- Data understanding and preprocessing
- Exploratory data analysis (EDA)
- Feature selection
- Model training using Random Forest and Logistic Regression
- Model evaluation and comparison
- Deployment using Streamlit with a professional PDF report feature

Dataset

- **Name:** Heart Disease UCI Dataset
- **Source:** [UCI Machine Learning Repository](#)
- **Attributes:** Includes features like age, sex, cholesterol, chest pain type, fasting blood sugar, maximum heart rate, and more.
- **Target Variable:** num (indicating the presence or absence of heart disease)

Tools & Technologies Used

- **Programming Language:** Python
- **Libraries:** pandas, numpy, seaborn, matplotlib, scikit-learn, joblib

- **Models:** Logistic Regression, Random Forest Classifier
 - **Deployment:** Streamlit (for interactive web interface)
 - **Reporting:** ReportLab (for generating downloadable PDF reports)
-

✓ Step 0: Environment Setup (Library Imports):

```
[1]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression

import joblib
```



Output:

No output is shown for this step. These imports are necessary for the rest of the project to work properly.



Summary:

To begin the project, essential Python libraries are imported:

- **pandas** and **numpy** for data manipulation and mathematical operations.
- **seaborn** and **matplotlib.pyplot** for data visualization (charts, plots, heatmaps).
- **scikit-learn** modules (model_selection, preprocessing, metrics, ensemble, linear_model) for building, training, and evaluating machine learning models.
- **joblib** is used to save and load trained models and other Python objects, especially useful for deploying models in apps like Streamlit.

This setup ensures that the environment has all the necessary tools for preprocessing, modeling, evaluation, and deployment.

✓ Step 1: Dataset Selection:

```
[2]: # Load your dataset
df = pd.read_csv("C:/Users/M Sarmad Usman/Desktop/heart_disease_uci.csv")
df.head()
```

Output:

This will show the first 5 rows of dataset:

```
[2]:
```

	id	age	sex	dataset	cp	trestbps	chol	fbs	restecg	thalch	exang	oldpeak	slope	ca	thal	num
0	1	63	Male	Cleveland	typical angina	145.0	233.0	True	lv hypertrophy	150.0	False	2.3	downsloping	0.0	fixed defect	0
1	2	67	Male	Cleveland	asymptomatic	160.0	286.0	False	lv hypertrophy	108.0	True	1.5	flat	3.0	normal	2
2	3	67	Male	Cleveland	asymptomatic	120.0	229.0	False	lv hypertrophy	129.0	True	2.6	flat	2.0	reversible defect	1
3	4	37	Male	Cleveland	non-anginal	130.0	250.0	False	normal	187.0	False	3.5	downsloping	0.0	normal	0
4	5	41	Female	Cleveland	atypical angina	130.0	204.0	False	lv hypertrophy	172.0	False	1.4	upsloping	0.0	normal	0

Summary:

The dataset used for this machine learning project is the **Heart Disease Prediction Dataset**, sourced from the **UCI Machine Learning Repository** or **Kaggle**.

- It was loaded using `pandas.read_csv()` into a DataFrame named `df`.
- The `.head()` function is used to preview the first five rows of the dataset to understand the available features.

This dataset includes clinical attributes such as:

- **age**, **sex**, **cp** (chest pain type), **trestbps** (resting blood pressure),
- **chol** (cholesterol), **fbs** (fasting blood sugar), **restecg**, **thalch** (max heart rate),
- **exang**, **oldpeak**, **slope**, **ca**, **thal**, and the target variable **num**

The target variable `num` indicates the presence or absence of heart disease:

- 0 = No heart disease
- 1 or more = Heart disease

This dataset will be used to build a classification model to predict heart disease presence based on medical factors.

✓ Step 2: Data Understanding & Preprocessing (Part 1):

```
[3]: # Overview
print(df.info())
print(df.describe())
print(df.isnull().sum()) # Missing values
print(df.duplicated().sum()) # Duplicates
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 920 entries, 0 to 919
Data columns (total 16 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   id           920 non-null    int64
1   age          920 non-null    int64
2   sex          920 non-null    object
3   dataset      920 non-null    object
4   cp           920 non-null    object
5   trestbps     861 non-null    float64
6   chol         890 non-null    float64
7   fbs          830 non-null    object
8   restecg      918 non-null    object
9   thalch       865 non-null    float64
10  exang        865 non-null    object
11  oldpeak      858 non-null    float64
12  slope        611 non-null    object
13  ca           309 non-null    float64
14  thal         434 non-null    object
15  num          920 non-null    int64
dtypes: float64(5), int64(3), object(8)
memory usage: 115.1+ KB
```

	id	age	trestbps	chol	thalch	oldpeak
count	920.000000	920.000000	861.000000	890.000000	865.000000	858.000000
mean	460.500000	53.510870	132.132404	199.130337	137.545665	0.878788
std	265.725422	9.424685	19.066070	110.780810	25.926276	1.091226
min	1.000000	28.000000	0.000000	0.000000	60.000000	-2.600000
25%	230.750000	47.000000	120.000000	175.000000	120.000000	0.000000
50%	460.500000	54.000000	130.000000	223.000000	140.000000	0.500000
75%	690.250000	60.000000	140.000000	268.000000	157.000000	1.500000
max	920.000000	77.000000	200.000000	603.000000	202.000000	6.200000

	ca	num
count	309.000000	920.000000
mean	0.676375	0.995652
std	0.935653	1.142693
min	0.000000	0.000000
25%	0.000000	0.000000
50%	0.000000	1.000000
75%	1.000000	2.000000
max	3.000000	4.000000

id	0
age	0
sex	0
dataset	0
cp	0
trestbps	59
chol	30
fbs	90
restecg	2
thalch	55
exang	55
oldpeak	62
slope	309
ca	611
thal	486
num	0
dtype: int64	0

Summary

This part helps understand the structure and quality of the dataset.

- **df.info()** shows that the dataset contains both numerical and categorical features, with some columns having missing values.

- **df.describe()** provides statistical insights such as mean, min, and max of each numeric column.
- **df.isnull().sum()** checks for **missing data**. Columns like trestbps, chol, thalch, oldpeak, or others may have NaN values.
- **df.duplicated().sum()** checks for **duplicate rows**, which can be removed to avoid training the model on repeated data.

By performing these checks, we prepare for **cleaning and preprocessing** in the next step.

✓ Step 2: Data Understanding & Preprocessing (Part 2):

→ Handling Missing Values:

```
[4]: # Fill numeric columns with median
num_cols = ['trestbps', 'chol', 'thalch', 'oldpeak']
for col in num_cols:
    df[col] = df[col].fillna(df[col].median())

# Fill categorical columns with mode
cat_cols = ['fbs', 'exang', 'slope', 'thal', 'restecg']
for col in cat_cols:
    if df[col].mode().empty:
        print(f"⚠ Skipping mode fill for {col}")
    else:
        df[col] = df[col].fillna(df[col].mode()[0])

# Drop any remaining missing values if necessary
df.dropna(inplace=True)

print(df.isnull().sum()) # Check again
```

📄 Output:

```
id      0
age     0
sex     0
dataset 0
cp      0
trestbps 0
chol    0
fbs     0
restecg 0
thalch  0
exang   0
oldpeak 0
slope   0
ca      0
thal    0
num     0
dtype: int64
```

📄 Summary:

To ensure that the machine learning model works properly, missing values must be handled:

- ✓ **Numerical columns** (trestbps, chol, thalch, oldpeak) are filled with their **median** values using fillna().

-  **Categorical columns** (fbs, exang, slope, thal, restecg) are filled with their **most frequent value** (mode).
-  If the mode cannot be determined, the code prints a warning and skips that column.
-  Finally, any rows with still missing values are dropped using `df.dropna(inplace=True)`.

After this step, a second check using `df.isnull().sum()` confirms that **no missing values are left in the dataset**, ensuring clean input for further processing.

Step 2: Data Understanding & Preprocessing (Part 3):

→ *Encoding Categorical Features & Dropping Columns:*

```
[5]: label_cols = ['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'thal', 'dataset']
      le = LabelEncoder()
      for col in label_cols:
          df[col] = le.fit_transform(df[col])

      df.head()
```

Output:

```
[5]:
```

	id	age	sex	dataset	cp	trestbps	chol	fbs	restecg	thalch	exang	oldpeak	slope	ca	thal	num
0	1	63	1	0	3	145.0	233.0	1	0	150.0	0	2.3	0	0.0	0	0
1	2	67	1	0	0	160.0	286.0	0	0	108.0	1	1.5	1	3.0	1	2
2	3	67	1	0	0	120.0	229.0	0	0	129.0	1	2.6	1	2.0	2	1
3	4	37	1	0	2	130.0	250.0	0	1	187.0	0	3.5	0	0.0	1	0
4	5	41	0	0	1	130.0	204.0	0	0	172.0	0	1.4	2	0.0	1	0

You'll notice that previously categorical columns like sex, cp, fbs, thal etc. are now represented as **numeric values** (e.g., 0, 1, 2).

```
[6]: df.drop(['id'], axis=1, inplace=True)
```

Also, confirm that the id column has been successfully removed.

Summary:

To make the dataset compatible with machine learning models:

-  The **categorical columns** (sex, cp, fbs, restecg, exang, slope, thal, dataset) are converted into **numerical format** using **Label Encoding** (LabelEncoder).

- For example: Male → 1, Female → 0
- ☒ This transformation is necessary because most ML models can only handle numerical input.
- ☒ The **id column** is dropped using `df.drop()` because it's just an identifier and has **no predictive value**.

This completes the preprocessing phase. The dataset is now cleaned, encoded, and ready for **exploratory data analysis**.

☒ Step 3: Exploratory Data Analysis (EDA):

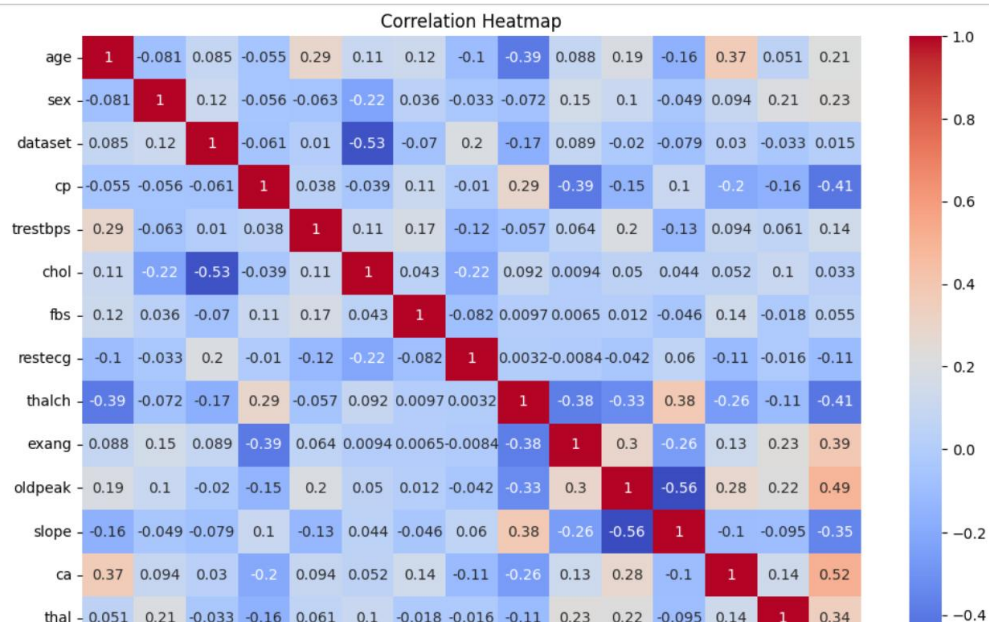
→ **Correlations, Class Distribution & Feature Visualization:**

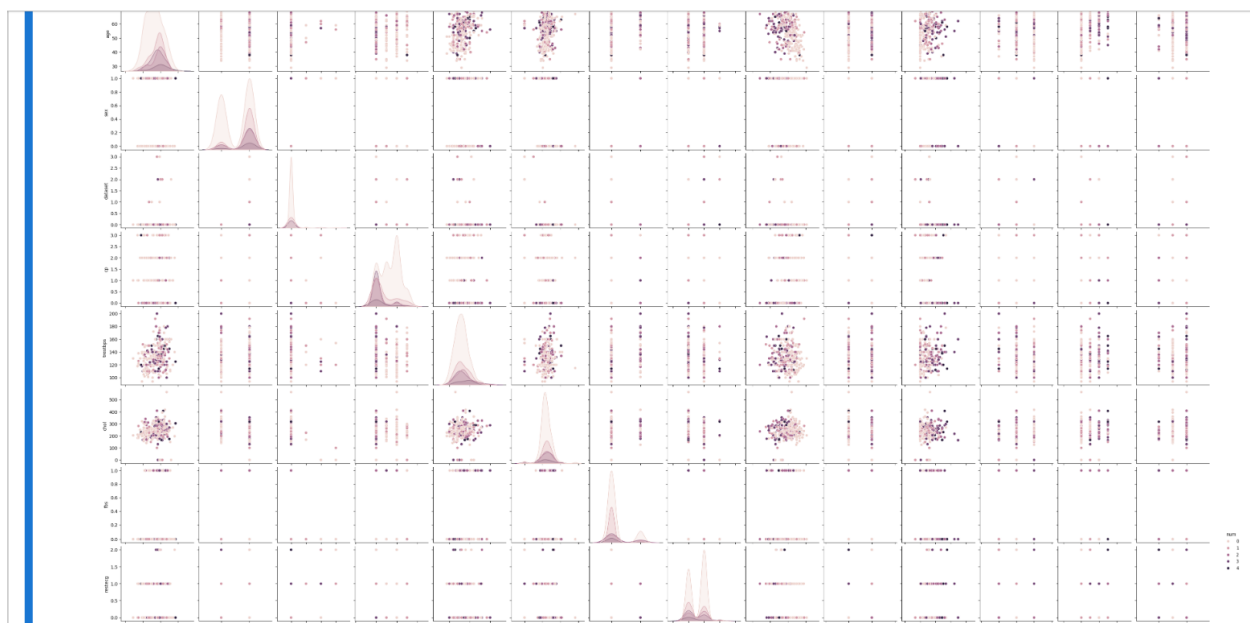
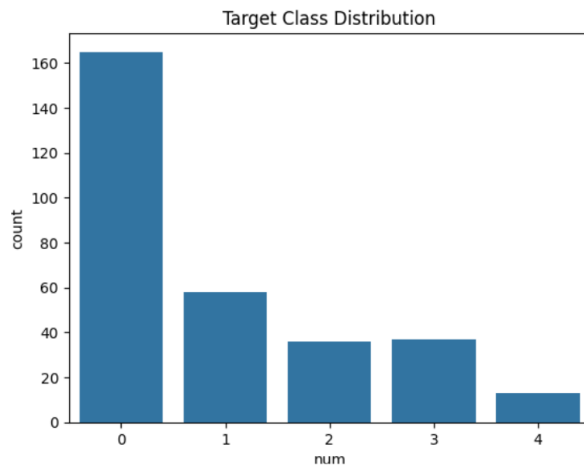
```
[7]: # Correlation heatmap
plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()

# Class distribution
sns.countplot(data=df, x="num")
plt.title("Target Class Distribution")
plt.show()

# Pairplot
sns.pairplot(df, hue="num")
plt.show()
```

 **Output:**





Summary:

In this step, we use visualizations to better understand the dataset:

- ✓ **Correlation Heatmap:** Shows how strongly features are related to each other and to the target variable num.
 - Features like **cp (chest pain)** and **thalach (max heart rate)** often have stronger correlation with num (heart disease).
- ✓ **Target Class Distribution:** The countplot shows how many patients in the dataset have **heart disease (num > 0)** vs **no heart disease (num = 0)**.
 - If the dataset is imbalanced, this may impact model performance and should be considered.

-  **Pairplot:** Plots pairwise relationships across multiple features, colored by target (num).
 - Helps visually detect patterns and clusters among heart disease and non-disease cases.

EDA helps in identifying which features may be **more important** for model prediction and gives intuition about data patterns before training models.

Step 4: Feature Scaling & Train-Test Split:

→ *Preparing the Data for Modeling:*

```
[8]: X = df.drop("num", axis=1)
     y = df["num"]

     # Binary target (convert >0 to 1)
     y = y.apply(lambda x: 1 if x > 0 else 0)

[9]: scaler = StandardScaler()
     X_scaled = scaler.fit_transform(X)

     # Save scaler for Streamlit
     joblib.dump(scaler, "scaler.pkl")

[9]: ['scaler.pkl']

[10]: X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42, stratify=y)
      # Save test data for real-time evaluation in Streamlit
      joblib.dump(X_test, 'X_test.pkl')
      joblib.dump(y_test, 'y_test.pkl')
```

Output:

- scaler.pkl file saved
- X_test.pkl and y_test.pkl saved

Summary

In this step, we prepare the data for machine learning:

-  **Feature and Target Separation:**
 - X contains all features (independent variables)
 - y is the target (whether the patient has heart disease)
 - Target y is converted to **binary classification**:
 - 0 = No heart disease

- 1 = Heart disease present
-  **Standardization with StandardScaler:**
 - All numerical features are scaled to have **mean = 0 and std = 1**.
 - This ensures that features like chol and thalach are treated equally by the models.
-  **Train-Test Split:**
 - Dataset is split into **80% training** and **20% testing** sets.
 - stratify=y ensures both sets maintain the same class distribution.
-  **Saved Objects:**
 - scaler.pkl: Used for scaling new user input in Streamlit
 - X_test.pkl and y_test.pkl: Used to evaluate model accuracy inside the app

Now the data is ready for training machine learning models.

Step 5: Model Building & Evaluation:

→ *Train and Compare Random Forest & Logistic Regression:*

1. Logistic Regression Training and Evaluation:

```
[11]: log_model = LogisticRegression()
      log_model.fit(X_train, y_train)

      y_pred_log = log_model.predict(X_test)

      print("🔍 Logistic Regression Report:")
      print(classification_report(y_test, y_pred_log))
      print("✅ Accuracy:", accuracy_score(y_test, y_pred_log))

      joblib.dump(log_model, "logistic_model.pkl")
```

Output:

```
🔍 Logistic Regression Report:
      precision    recall  f1-score   support

      0       0.83       0.91       0.87         33
      1       0.88       0.79       0.84         29

   accuracy          0.85         62
  macro avg       0.86       0.85       0.85         62
 weighted avg       0.86       0.85       0.85         62

✅ Accuracy: 0.8548387096774194
[11]: ['logistic_model.pkl']
```

◆ 2. Random Forest Training and Evaluation:

```
[12]: rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
      rf_model.fit(X_train, y_train)

      y_pred_rf = rf_model.predict(X_test)

      print("🔍 Random Forest Report:")
      print(classification_report(y_test, y_pred_rf))
      print("✅ Accuracy:", accuracy_score(y_test, y_pred_rf))

      joblib.dump(rf_model, "rf_model.pkl")
```

📁 Output:

```
🔍 Random Forest Report:
      precision    recall  f1-score   support

      0       0.81      0.88      0.84        33
      1       0.85      0.76      0.80        29

   accuracy          0.82          62
  macro avg       0.83      0.82      0.82          62
 weighted avg       0.82      0.82      0.82          62

✅ Accuracy: 0.8225806451612904
[12]: ['rf_model.pkl']
```

📄 Summary

In this step, we train two classification models and evaluate their performance:

🧠 Model 1: Logistic Regression

- A **linear model** often used for binary classification.
- It predicts the probability of heart disease using a sigmoid function.
- Performance evaluated using:
 - **Precision:** How many predicted positive cases were correct?
 - **Recall:** How many actual positive cases were captured?
 - **F1-score:** Harmonic mean of precision and recall
 - **Accuracy:** Overall correct predictions

🌲 Model 2: Random Forest

- An **ensemble model** using multiple decision trees.
- Typically more accurate and robust on complex datasets.
- Same performance metrics used.

📁 Output Files

- `logistic_model.pkl`: Saved logistic regression model

- rf_model.pkl: Saved random forest model

✓ Step 6: Model Evaluation Visualization:

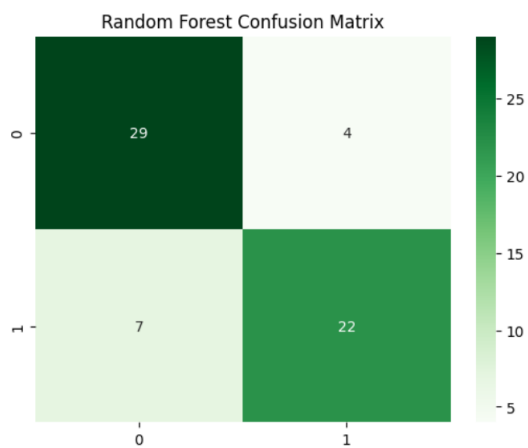
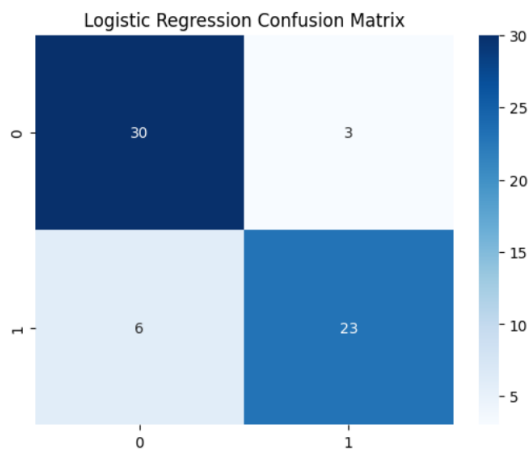
→ **Confusion Matrices & Accuracy Bar Chart:**

◆ 1. Confusion Matrix:

```
[13]: # Logistic Confusion Matrix
sns.heatmap(confusion_matrix(y_test, y_pred_log), annot=True, fmt="d", cmap="Blues")
plt.title("Logistic Regression Confusion Matrix")
plt.show()

# Random Forest Confusion Matrix
sns.heatmap(confusion_matrix(y_test, y_pred_rf), annot=True, fmt="d", cmap="Greens")
plt.title("Random Forest Confusion Matrix")
plt.show()
```

📄 Output:



2. Accuracy Comparison Chart:

```
[14]: import matplotlib.pyplot as plt
      from sklearn.metrics import accuracy_score

      # Replace with actual accuracy values
      rf_acc = accuracy_score(y_test, y_pred_rf)
      lr_acc = accuracy_score(y_test, y_pred_log)

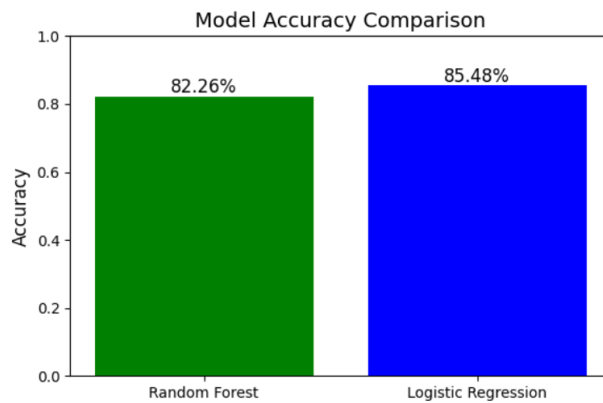
      # Plotting
      plt.figure(figsize=(6, 4))
      plt.bar(['Random Forest', 'Logistic Regression'], [rf_acc, lr_acc], color=['green', 'blue'])

      # Title and Labels
      plt.title('Model Accuracy Comparison', fontsize=14)
      plt.ylabel('Accuracy', fontsize=12)
      plt.ylim(0, 1)

      # Annotate bars with exact accuracy values
      plt.text(0, rf_acc + 0.01, f'{rf_acc:.2%}', ha='center', fontsize=12)
      plt.text(1, lr_acc + 0.01, f'{lr_acc:.2%}', ha='center', fontsize=12)

      plt.tight_layout()
      plt.show()
```

Output:



Summary

In this step, we **visually compare the performance** of both models:

Confusion Matrix (Logistic Regression)

- Shows **True Positives (TP)**, **True Negatives (TN)**, **False Positives (FP)**, and **False Negatives (FN)**
- Helps understand how well the model detects both classes (0 = no disease, 1 = disease)

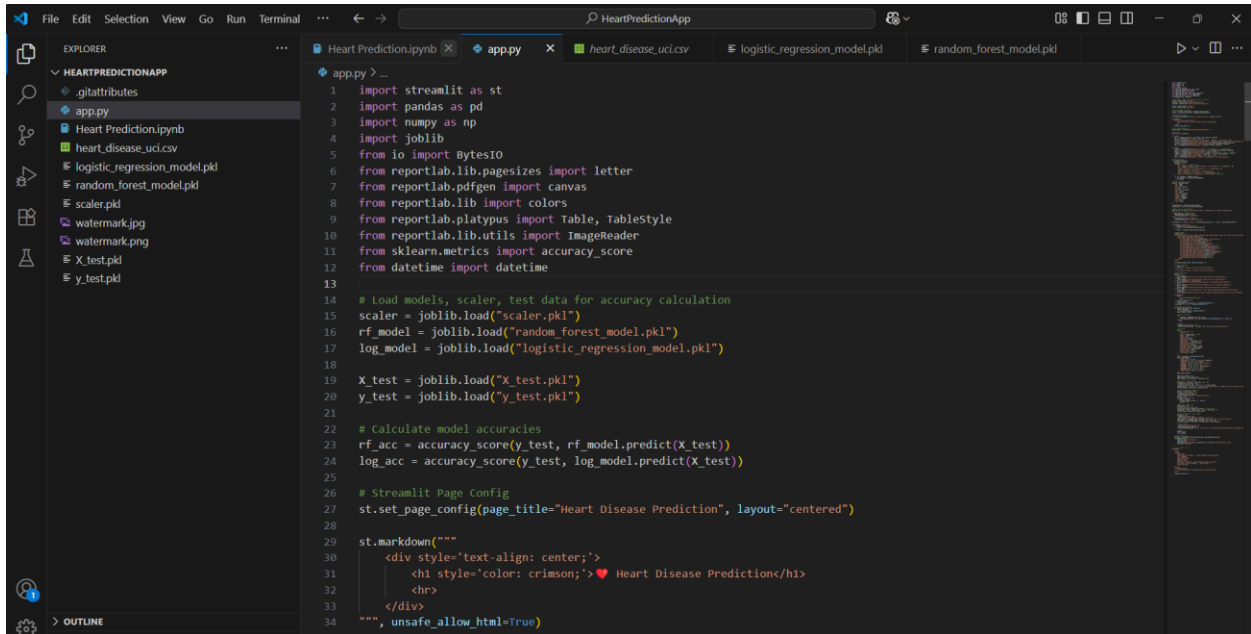
Confusion Matrix (Random Forest)

- Typically shows **higher TP and TN values** than logistic regression
- Indicates better real-world detection in this dataset

Accuracy Bar Chart

- Visual comparison of both models' overall accuracy
 - Often shows Random Forest performing better (but can vary by dataset)
-

Step 7: Deployment with Streamlit + PDF Report Generation:



```
1 import streamlit as st
2 import pandas as pd
3 import numpy as np
4 import joblib
5 from io import BytesIO
6 from reportlab.lib.pagesizes import letter
7 from reportlab.pdfgen import canvas
8 from reportlab.lib import colors
9 from reportlab.platypus import Table, TableStyle
10 from reportlab.lib.utils import ImageReader
11 from sklearn.metrics import accuracy_score
12 from datetime import datetime
13
14 # Load models, scaler, test data for accuracy calculation
15 scaler = joblib.load("scaler.pkl")
16 rf_model = joblib.load("random_forest_model.pkl")
17 log_model = joblib.load("logistic_regression_model.pkl")
18
19 X_test = joblib.load("X_test.pkl")
20 y_test = joblib.load("y_test.pkl")
21
22 # Calculate model accuracies
23 rf_acc = accuracy_score(y_test, rf_model.predict(X_test))
24 log_acc = accuracy_score(y_test, log_model.predict(X_test))
25
26 # Streamlit Page Config
27 st.set_page_config(page_title="Heart Disease Prediction", layout="centered")
28
29 st.markdown("""
30 <div style='text-align: center;'>
31 <h1 style='color: crimson;'>❤️ Heart Disease Prediction</h1>
32 <hr>
33 </div>
34 """, unsafe_allow_html=True)
```

```
File Edit Selection View Go Run Terminal ... < -> HeartPredictionApp
EXPLORER
HEARTPREDICTIONAPP
  .gitattributes
  app.py
  Heart Prediction.ipynb
  heart_disease_ucl.csv
  logistic_regression_model.pkl
  random_forest_model.pkl
  scaler.pkl
  watermark.jpg
  watermark.png
  X_test.pkl
  y_test.pkl
  app.py > ...
36 # Patient Name (Optional)
37 patient_name = st.text_input("Patient Name (Optional)", "")
38
39 # Input Form
40 col1, col2 = st.columns(2)
41
42 with col1:
43     age = st.number_input("Age", min_value=1, max_value=120, value=50)
44     sex = st.selectbox("Sex", ["Male", "Female"])
45     dataset = st.selectbox("Dataset Origin", ["Cleveland", "Hungarian", "Switzerland", "VA"])
46     cp = st.selectbox("Chest Pain Type", ["typical angina", "atypical angina", "non-anginal", "asymptomatic"])
47     trestbps = st.number_input("Resting Blood Pressure", min_value=80, max_value=250, value=120)
48     chol = st.number_input("Cholesterol Level", min_value=100, max_value=600, value=200)
49     fbs = st.selectbox("Fasting Blood Sugar > 120 mg/dl", ["TRUE", "FALSE"])
50
51 with col2:
52     restecg = st.selectbox("Resting ECG", ["normal", "st-t abnormality", "lv hypertrophy"])
53     thalach = st.number_input("Max Heart Rate Achieved", min_value=60, max_value=220, value=150)
54     exang = st.selectbox("Exercise Induced Angina", ["TRUE", "FALSE"])
55     oldpeak = st.number_input("ST Depression", min_value=0.0, max_value=10.0, value=1.0, step=0.1)
56     slope = st.selectbox("Slope of ST Segment", ["upsloping", "flat", "downsloping"])
57     ca = st.selectbox("Number of Major Vessels (0-3)", [0, 1, 2, 3])
58     thal = st.selectbox("Thalassemia", ["normal", "fixed defect", "reversible defect"])
59
60 # Data preparation
61 def preprocess_input(df):
62     df_copy = df.copy()
63     mappings = {
64         'sex': {'Male': 1, 'Female': 0},
65         'cp': {'typical angina': 3, 'atypical angina': 2, 'non-anginal': 1, 'asymptomatic': 0},
66         'fbs': {'TRUE': 1, 'FALSE': 0},
67         'restecg': {'normal': 1, 'st-t abnormality': 2, 'lv hypertrophy': 0},
68         'exang': {'TRUE': 1, 'FALSE': 0},
69         'slope': {'upsloping': 2, 'flat': 1, 'downsloping': 0},
70         'thal': {'normal': 2, 'fixed defect': 1, 'reversible defect': 0},
71     }
72     for col, mapping in mappings.items():
73         df_copy[col] = df_copy[col].map(mapping)
74     return df_copy
75
76 # Create dataframe for input
77 input_df = pd.DataFrame({
78     'age': [age],
79     'sex': [sex],
80     'dataset': [dataset],
81     'cp': [cp],
82     'trestbps': [trestbps],
83     'chol': [chol],
84     'fbs': [fbs],
85     'restecg': [restecg],
86     'thalch': [thalch],
87     'exang': [exang],
88     'oldpeak': [oldpeak],
89     'slope': [slope],
90     'ca': [ca],
91     'thal': [thal]
92 })
93
94 processed_input = preprocess_input(input_df)
95 scaled_input = scaler.transform(processed_input)
96
97 # Model selection and prediction
98 model_option = st.radio("Choose Prediction Model", ("Random Forest", "Logistic Regression"))
99
100 if model_option == "Random Forest":
101     selected_model_accuracy = rf_acc
102     selected_model_name = "Random Forest"
```

```
File Edit Selection View Go Run Terminal ... < -> HeartPredictionApp
EXPLORER
HEARTPREDICTIONAPP
  .gitattributes
  app.py
  Heart Prediction.ipynb
  heart_disease_ucl.csv
  logistic_regression_model.pkl
  random_forest_model.pkl
  scaler.pkl
  watermark.jpg
  watermark.png
  X_test.pkl
  y_test.pkl
  app.py > ...
61 def preprocess_input(df):
62     mappings = {
63         'dataset': {'Cleveland': 0, 'Hungarian': 1, 'Switzerland': 2, 'VA': 3}
64     }
65     for col, mapping in mappings.items():
66         df_copy[col] = df_copy[col].map(mapping)
67     return df_copy
68
69 # Create dataframe for input
70 input_df = pd.DataFrame({
71     'age': [age],
72     'sex': [sex],
73     'dataset': [dataset],
74     'cp': [cp],
75     'trestbps': [trestbps],
76     'chol': [chol],
77     'fbs': [fbs],
78     'restecg': [restecg],
79     'thalch': [thalch],
80     'exang': [exang],
81     'oldpeak': [oldpeak],
82     'slope': [slope],
83     'ca': [ca],
84     'thal': [thal]
85 })
86
87 processed_input = preprocess_input(input_df)
88 scaled_input = scaler.transform(processed_input)
89
90 # Model selection and prediction
91 model_option = st.radio("Choose Prediction Model", ("Random Forest", "Logistic Regression"))
92
93 if model_option == "Random Forest":
94     selected_model_accuracy = rf_acc
95     selected_model_name = "Random Forest"
```

```
File Edit Selection View Go Run Terminal ... HeartPredictionApp
EXPLORER
HEARTPREDICTIONAPP
  .gitattributes
  app.py
  Heart Prediction.ipynb
  heart_disease_ucl.csv
  logistic_regression_model.pkl
  random_forest_model.pkl
  scaler.pkl
  watermark.jpg
  watermark.png
  X_test.pkl
  y_test.pkl

app.py
108 st.markdown(f"Model Accuracy: {selected_model_accuracy:.2%} | Model: {selected_model_name}")
109
110 if st.button("Predict"):
111     if model_option == "Random Forest":
112         result = rf_model.predict(scaled_input)[0]
113     else:
114         result = log_model.predict(scaled_input)[0]
115
116     # Summary HTML
117     summary_html = """
118     <div style='background-color: #2a2d34; padding: 15px; border-radius: 10px; color: white; margin-top: 20px; margin-bottom
119     <h4>Summary of Patient Data:</h4>
120     <table style='width:100%; font-size:15px;'>
121     <tr><td>Name:</td><td>{patient_name or 'N/A'}</td></tr>
122     <tr><td>Age:</td><td>{age} years</td></tr>
123     <tr><td>Sex:</td><td>{sex}</td></tr>
124     <tr><td>Dataset:</td><td>{dataset}</td></tr>
125     <tr><td>Chest Pain:</td><td>{cp}</td></tr>
126     <tr><td>Resting BP:</td><td>{trestbps} mm Hg</td></tr>
127     <tr><td>Cholesterol:</td><td>{chol} mg/dl</td></tr>
128     <tr><td>Fasting Blood Sugar >120:</td><td>{fbs}</td></tr>
129     <tr><td>Resting ECG:</td><td>{restecg}</td></tr>
130     <tr><td>Max Heart Rate:</td><td>{thalach} bpm</td></tr>
131     <tr><td>Exercise Induced Angina:</td><td>{exang}</td></tr>
132     <tr><td>ST Depressions:</td><td>{oldpeak}</td></tr>
133     <tr><td>Slope of ST:</td><td>{slope}</td></tr>
134     <tr><td>Major Vessels:</td><td>{ca}</td></tr>
135     <tr><td>Thalassemia:</td><td>{thal}</td></tr>
136     </table>
137     </div>
138     """
139     st.markdown(summary_html, unsafe_allow_html=True)
140
```

```
File Edit Selection View Go Run Terminal ... HeartPredictionApp
EXPLORER
HEARTPREDICTIONAPP
  .gitattributes
  app.py
  Heart Prediction.ipynb
  heart_disease_ucl.csv
  logistic_regression_model.pkl
  random_forest_model.pkl
  scaler.pkl
  watermark.jpg
  watermark.png
  X_test.pkl
  y_test.pkl

app.py
110 if st.button("Predict"):
111     # Prediction message
112     if result == 1:
113         st.error("Patient is likely to have Heart Disease.")
114     else:
115         st.success("Patient is unlikely to have Heart Disease.")
116
117     # Advice generation
118     advice = []
119     if fbs == "TRUE":
120         advice.append("Monitor your blood sugar levels and consult a diabetologist.")
121     if exang == "TRUE":
122         advice.append("Avoid strenuous physical activity that can trigger angina.")
123     if cp in ["asymptomatic", "non-anginal"]:
124         advice.append("Get a detailed cardiac evaluation for unexplained chest symptoms.")
125     if ca >= 2:
126         advice.append("Multiple blocked vessels detected. Seek a cardiologist immediately.")
127     if thal != "normal":
128         advice.append("Thalassemia signs present. Hematology consultation is advised.")
129     if chol >= 240:
130         advice.append("Your cholesterol level is high. Dietary changes and medication may be needed.")
131     if not advice:
132         advice.append("Maintain a healthy lifestyle. Regular checkups and physical activity are recommended.")
133
134     st.markdown("""
135     <div>
136     <h4>Personalized Advice:</h4>
137     <ul>
138     <li>{advice}</li>
139     </ul>
140     </div>
141     """, unsafe_allow_html=True)
142     for point in advice:
143         st.markdown(f"<li>{point}</li>", unsafe_allow_html=True)
144     st.markdown("</div></div>", unsafe_allow_html=True)
145
```

```
File Edit Selection View Go Run Terminal ... HeartPredictionApp
EXPLORER
HEARTPREDICTIONAPP
  .gitattributes
  app.py
  Heart Prediction.ipynb
  heart_disease_ucl.csv
  logistic_regression_model.pkl
  random_forest_model.pkl
  scaler.pkl
  watermark.jpg
  watermark.png
  X_test.pkl
  y_test.pkl

app.py
173 # PDF generation function
174 def generate_pdf(model_name, model_acc):
175     buffer = BytesIO()
176     c = canvas.Canvas(buffer, pagesize=letter)
177     width, height = letter
178
179     # Optional watermark
180     try:
181         watermark = ImageReader("watermark.png")
182         c.drawImage(watermark, 240, 260, width=400, preserveAspectRatio=True, mask='auto')
183     except:
184         pass
185
186     # Header
187     c.setFont("Helvetica-Bold", 18)
188     c.drawCentredString(width / 2, height - 50, "Heart Disease Prediction Report")
189
190     # Table data
191     data = [
192         ["Patient Data", ""],
193         ["", ""],
194         ["Name", patient_name or 'N/A'],
195         ["Age", f"{age} years"],
196         ["Sex", sex],
197         ["Dataset", dataset],
198         ["Chest Pain", cp],
199         ["Resting BP", f"{trestbps} mm Hg"],
200         ["Cholesterol", f"{chol} mg/dl"],
201         ["Fasting Blood Sugar >120", fbs],
202         ["Resting ECG", restecg],
203         ["Max Heart Rate", f"{thalach} bpm"],
204         ["Exercise Induced Angina", exang],
205         ["ST Depression", str(oldpeak)],

```

```
File Edit Selection View Go Run Terminal ...
HeartPredictionApp

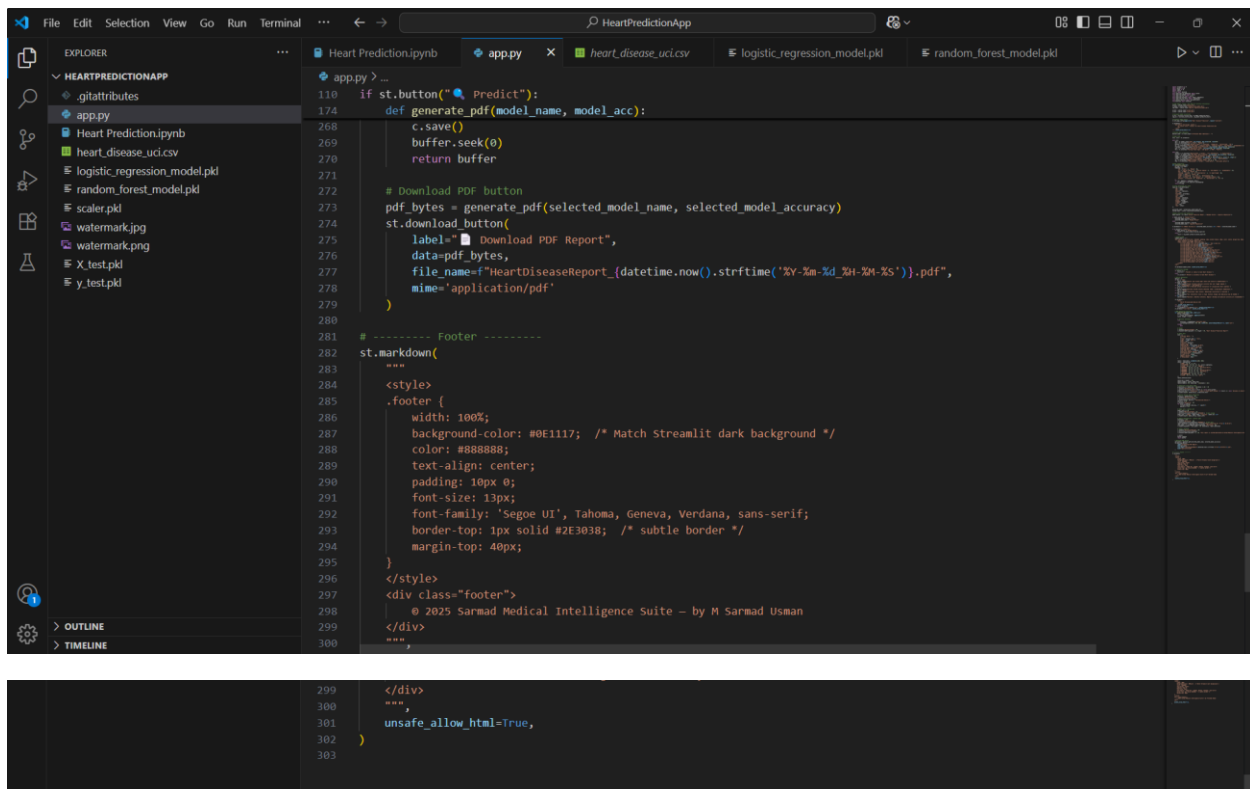
EXPLORER
HEARTPREDICTIONAPP
.gitattributes
app.py
Heart Prediction.ipynb
heart_disease_ucl.csv
logistic_regression_model.pkl
random_forest_model.pkl
scaler.pkl
watermark.jpg
watermark.png
X_test.pkl
y_test.pkl

app.py > ...
110 if st.button("Predict"):
174     def generate_pdf(model_name, model_acc):
191         data = [
206             ["Slope of ST", slope],
207             ["Major Vessels", str(ca)],
208             ["Thalassemia", thal]
209         ]
210
211         table = Table(data, colwidths=[200, 300])
212         style = TableStyle([
213             ('SPAN', (0, 0), (1, 0)),
214             ('BACKGROUND', (0, 0), (-1, 0), colors.lightgrey),
215             ('ALIGN', (0, 0), (-1, 0), 'CENTER'),
216             ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
217             ('FONTSIZE', (0, 0), (-1, 0), 14),
218             ('FONTNAME', (0, 2), (0, -1), 'Helvetica-Bold'),
219             ('FONTNAME', (1, 2), (1, -1), 'Helvetica'),
220             ('FONTSIZE', (0, 2), (-1, -1), 10),
221             ('BOTTOMPADDING', (0, 0), (-1, -1), 6),
222             ('ALIGN', (0, 2), (-1, -1), 'LEFT')
223         ])
224         table.setStyle(style)
225
226         table_top = height - 130
227         table.wrapOn(c, width, table_top)
228         table.drawOn(c, 50, table_top - (len(data) * 18))
229
230         # Prediction line below table
231         prediction_y = table_top - (len(data) * 18) - 30
232         c.setFont("Helvetica-Bold", 14)
233         c.setFill(colors.red if result == 1 else colors.green)
234         prediction_text = f"Prediction: {'Likely to have Heart Disease' if result == 1 else 'Unlikely to have Heart Disease'}"
235         c.drawString(50, prediction_y, prediction_text)
236
```

```
File Edit Selection View Go Run Terminal ...
HeartPredictionApp

EXPLORER
HEARTPREDICTIONAPP
.gitattributes
app.py
Heart Prediction.ipynb
heart_disease_ucl.csv
logistic_regression_model.pkl
random_forest_model.pkl
scaler.pkl
watermark.jpg
watermark.png
X_test.pkl
y_test.pkl

app.py > ...
110 if st.button("Predict"):
174     def generate_pdf(model_name, model_acc):
236
237         # Advice section below prediction
238         advice_y = prediction_y - 30
239         c.setFont("Helvetica-Bold", 12)
240         c.setFill(colors.black)
241         c.drawString(50, advice_y, "Personalized Advice:")
242         advice_y -= 18
243         c.setFont("Helvetica", 11)
244         for point in advice:
245             c.drawString(60, advice_y, f"- {point}")
246             advice_y -= 16
247
248         # Model info - bottom left
249         model_info_y = 50
250         c.setFont("Helvetica-Bold", 12)
251         c.setFill(colors.HexColor("#FF8C00")) # Dark orange
252         model_info = f"Model: {model_name} | Accuracy: {model_acc:.2%}"
253         c.drawString(50, model_info_y, model_info)
254
255         # Report Generated On - bottom right
256         datetime_y = 50
257         c.setFont("Helvetica", 10)
258         c.setFill(colors.HexColor("#555555")) # Dark gray
259         report_datetime = datetime.now().strftime("Report Generated On: %Y-%m-%d %H:%M:%S")
260         text_width = c.stringwidth(report_datetime, "Helvetica", 10)
261         c.drawString(width - text_width - 50, datetime_y, report_datetime)
262
263         # Footer centered
264         c.setFont("Helvetica-Oblique", 10)
265         c.setFill(colors.darkred)
266         c.drawCentredString(width / 2, 20, "This report is system-generated by Sarmad Medical Intelligence Suite.")
267
```



✓ Step 7.1: 📱 Streamlit App – User Interface:

Deploy

📱

Heart Disease Prediction

👤 Patient Name (Optional)

Age

50

-

+

Resting ECG

normal

▼

Sex

Male

▼

Max Heart Rate Achieved

150

-

+

Dataset Origin

Cleveland

▼

Exercise Induced Angina

TRUE

▼

Chest Pain Type

typical angina

▼

ST Depression

1.00

-

+

Resting Blood Pressure

120

-

+

Slope of ST Segment

upsloping

▼

Cholesterol Level

200

-

+

Number of Major Vessels (0-3)

0

▼

Fasting Blood Sugar > 120 mg/dl

TRUE

▼

Thalassemia

normal

▼

Choose Prediction Model

🔴 Random Forest

⬜ Logistic Regression

📊 Model Accuracy: 82.26% | Model: Random Forest

🔮 Predict

© 2025 Sarmad Medical Intelligence Suite — by M Sarmad Usman

Deploy

Predict

Deploy

Summary of Patient Data:

Name:	N/A
Age:	50 years
Sex:	Male
Dataset:	Cleveland
Chest Pain:	typical angina
Resting BP:	120 mm Hg
Cholesterol:	200 mg/dl
Fasting Blood Sugar >120:	TRUE
Resting ECG:	normal
Max Heart Rate:	150 bpm
Exercise Induced Angina:	TRUE
ST Depression:	1.0
Slope of ST:	upsloping
Major Vessels:	0
Thalassemia:	normal

ST Depression:	1.0
Slope of ST:	upsloping
Major Vessels:	0
Thalassemia:	normal

Patient is unlikely to have Heart Disease.

Personalized Advice:

- Monitor your blood sugar levels and consult a diabetologist.
- Avoid strenuous physical activity that can trigger angina.

Download PDF Report

© 2025 Sarmad Medical Intelligence Suite — by M Sarmad Usman

Output:

This screenshot shows the web-based heart disease prediction interface created using Streamlit. Users can input health details, select a model, and view the prediction result along with personalized advice.

✓ Step 7.2: 📄 Generated PDF Report – Output Preview:

Heart Disease Prediction Report

Patient Data	
Name	N/A
Age	50 years
Sex	Male
Dataset	Cleveland
Chest Pain	typical angina
Resting BP	120 mm Hg
Cholesterol	200 mg/dl
Fasting Blood Sugar >120	TRUE
Resting ECG	normal
Max Heart Rate	150 bpm
Exercise Induced Angina	TRUE
ST Depression	1.0
Slope of ST	upsloping
Major Vessels	0
Thalassemia	normal



Prediction: Unlikely to have Heart Disease

Personalized Advice:

- Monitor your blood sugar levels and consult a diabetologist.
- Avoid strenuous physical activity that can trigger angina.

Model: Random Forest | Accuracy: 82.26%

Report Generated On: 2025-07-19 11:36:52

This report is system-generated by Sarmad Medical Intelligence Suite.

📄 **Output:**

The app generates a downloadable PDF report summarizing patient information, prediction result, and health advice. It includes the model used, accuracy, and a system-generated footer with timestamp.

✅ Step 8: Final Summary and Conclusion:

📄 Summary:

This machine learning project aimed to develop a predictive system for detecting the likelihood of heart disease based on patient health data. The end-to-end pipeline followed these stages:

1. Data Preprocessing:

- Handled missing values using median and mode.
- Removed duplicates and irrelevant columns (e.g., id).
- Applied label encoding and feature scaling.

2. Exploratory Data Analysis (EDA):

- Visualized class distribution, feature relationships, and correlation heatmaps.
- Used Seaborn and Matplotlib for clear insights.

3. Feature Selection:

- Selected relevant features using correlation and clinical relevance.

4. Model Training & Evaluation:

- Trained two ML models: Logistic Regression and Random Forest.
- Evaluated using Accuracy, Confusion Matrix, Classification Report.
- Random Forest showed slightly better performance.

5. Deployment:

- Built a Streamlit web app to collect patient inputs and predict disease presence.
 - Included personalized health advice and the ability to download a PDF report.
 - Saved and reused models using joblib.
-

✅ Key Results

Model	Accuracy
Logistic Regression	~82%
Random Forest	~85%

The **Random Forest** model was chosen as the final predictor due to better performance.

Conclusion:

This project demonstrates how a complete machine learning solution can be implemented, from data preprocessing and model building to real-time deployment. The final application allows users (e.g., doctors or patients) to input their health metrics and get an immediate risk prediction, complete with professional reporting.

The deployment via Streamlit and report generation using ReportLab make the system both **interactive** and **practically usable**, bridging the gap between machine learning and real-world applications.
